

Data Parallelism Implementation

DDP stands for Distributed Data Parallel, a training wrapper that replicates a model on multiple devices and averages gradients across them via collectives like all-reduce. After synchronization, each replica applies the same optimizer step so all copies stay in lockstep for correct, scalable training.

Steps

1. Deterministic, non-overlapping data sharding per rank
2. Forward + backward per device independently
3. Collective gradient synchronization (all-reduce)
4. Identical parameter updates on all devices

Good practices

- Use a seeded `DistributedSampler` for dataloaders
- Overlap communication and compute via gradient bucketing
- Configure DDP for unused params when models branch (`find_unused_parameters`)
- Gradient clipping before the optimizer step
- Mixed precision: prefer BF16 if available; otherwise FP16 with dynamic loss scaling

Notes on synchronization

- Ring all-reduce minimizes bandwidth and scales with $N-1$ hops
- Ensure optimizer steps are identical across ranks after sync

Scalability Analysis

Sources of scaling loss

- Communication overhead: bandwidth limits, latency, protocol inefficiency
- Synchronization barriers and stragglers: everyone waits for the slowest
- Load imbalance/heterogeneity: uneven work, throttling, ECC, I/O skew
- Memory pressure: activations/optimizer state constrain batch/parallelism

Measuring efficiency

- Ideal speedup = N
- Actual speedup = T_1 / T_N
- Parallel efficiency = $(T_1 / T_N) / N$

Amdahl's Law (upper bound on speedup)

- Speedup $\leq 1 / (s + (1 - s)/N)$, where s = serial fraction
- Example: $s = 0.10 \Rightarrow \text{max speedup} = 10\times \text{ as } N \rightarrow \infty$
- Even small s severely caps efficiency at large N

What to do (reduce s , hide comm)

- Overlap communication with compute: gradient bucketing, fused kernels
- Hierarchical collectives; topology-aware placement
- Lower-precision or compressed gradients with error feedback
- Algorithmic options: increase local batch; partial/delayed sync; asynchronous variants with care
- Balance work: uniform microbatching, stage rebalancing, dynamic batch assignment
- Detect and mitigate stragglers: health checks, retry, prefetch, pinned workers

Sync vs Async vs Elastic Data Parallel

Synchronous all-reduce:

- Pros: stable convergence; single-node equivalence
- Cons: straggler-sensitive; per-step barrier

Parameter-server (async/stale-sync):

- Pros: more throughput; tolerant to slow clients
- Cons: stale gradients; careful LR tuning; potential accuracy hit

Elastic DP (synchronous with changing world size):

- World size adjusts at runtime; reshard data/checkpoints; adjust LR with effective batch

Guidance: prefer sync for accuracy-critical; use async/elastic when churn is inevitable